



LISTA DE EXERCÍCIOS – FUNDAMENTOS DE SISTEMAS INTELIGENTES

Lista de Exercícios 2

DOCENTE: Thiago França Naves

DATA: ____ / ____ / ____

ALUNO: Leonardo José Reis Pinto

OBS: Durante o período de APNP todos os exercícios resolvidos devem conter além do desenvolvimento da técnica ou do algoritmo de resposta, também um resumo de como você chegou na solução, suas partes e relação com o algoritmo/técnica desenvolvido. Esse será um dos principais critérios de avaliação da resposta e atribuição de nota a lista.

1. Considere o seguinte grafo “dirigido” (mapa). O nó A representa o estado inicial e o nó G representa o objetivo a ser alcançado. As ações permitidas são representadas pelos arcos de cada nó. O custo do caminho de um nó para outro está indicado pelo número associado a cada. O custo estimado (via alguma função heurística) de cada nó em relação ao nó objetivo está indicado pelo número dentro de cada círculo representando o nó
A. Desenhe a árvore de busca para este grafo. Coloque os nós em ordem alfabética da esquerda para a direita. Se quiser, adicione o custo do caminho de cada arco, como também o valor da função heurística para cada nó (isto irá ajudar na solução dos próximos itens).

R:

Os valores da heurísticos são:

$h(A)=30$

$h(B)=26$

$h(C)=21$

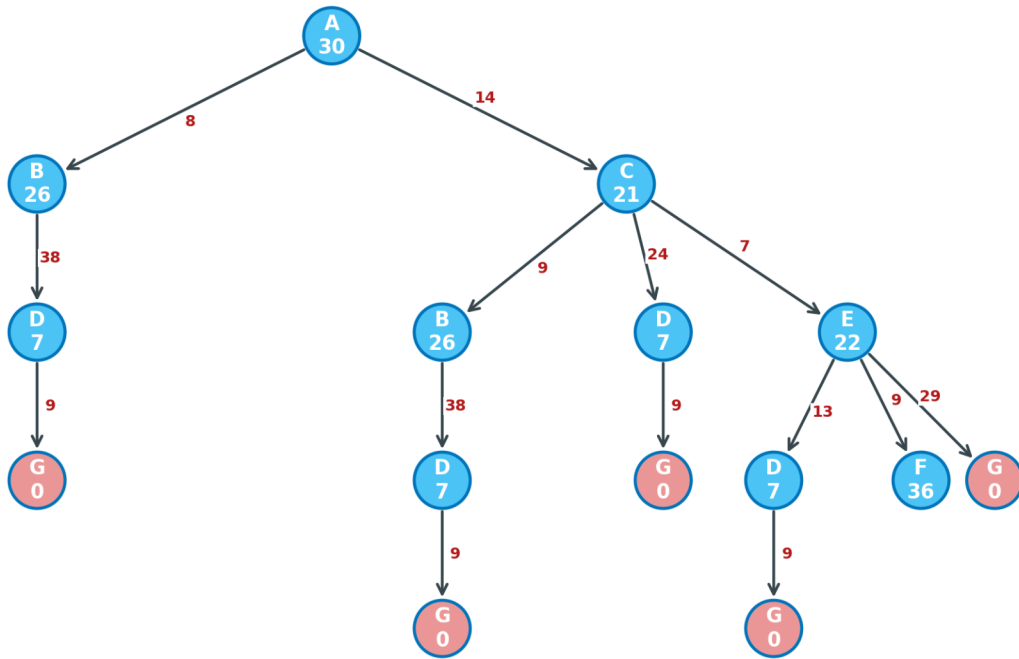
$h(D)=7$

$h(E)=22$

$h(F)=36$

$h(G)=0$

O nó inicial é A e o objetivo é G.



B. Qual o caminho ótimo do nó inicial para o nó objetivo?

R: O caminho ótimo é o de menor custo total, não o de menos passos.

Comparando todos os caminhos de A até G:

A->B->D->G $8+38+9 = 55$

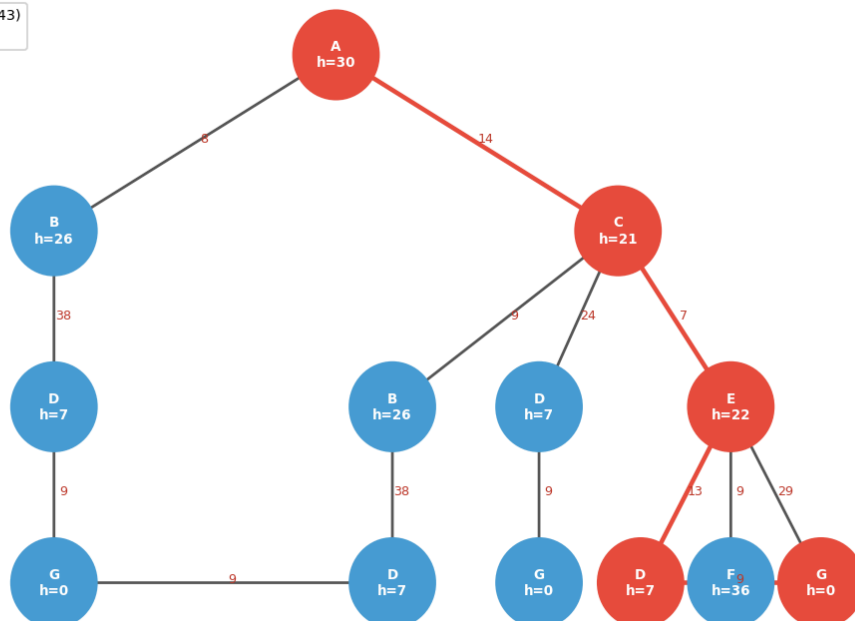
A->C->D->G $14+24+9 = 47$

A->C->E->G $14+7+29 = 50$

A->C->E->D->G: $14+7+13+9 = 43$, Portanto, o caminho ótimo é A -> C -> E -> D -> G com custo = 43.

Q1B — Árvore de Busca: Caminho Ótimo A→C→E→D→G (custo=43)

■ Caminho ótimo (custo=43)
■ Outros nós

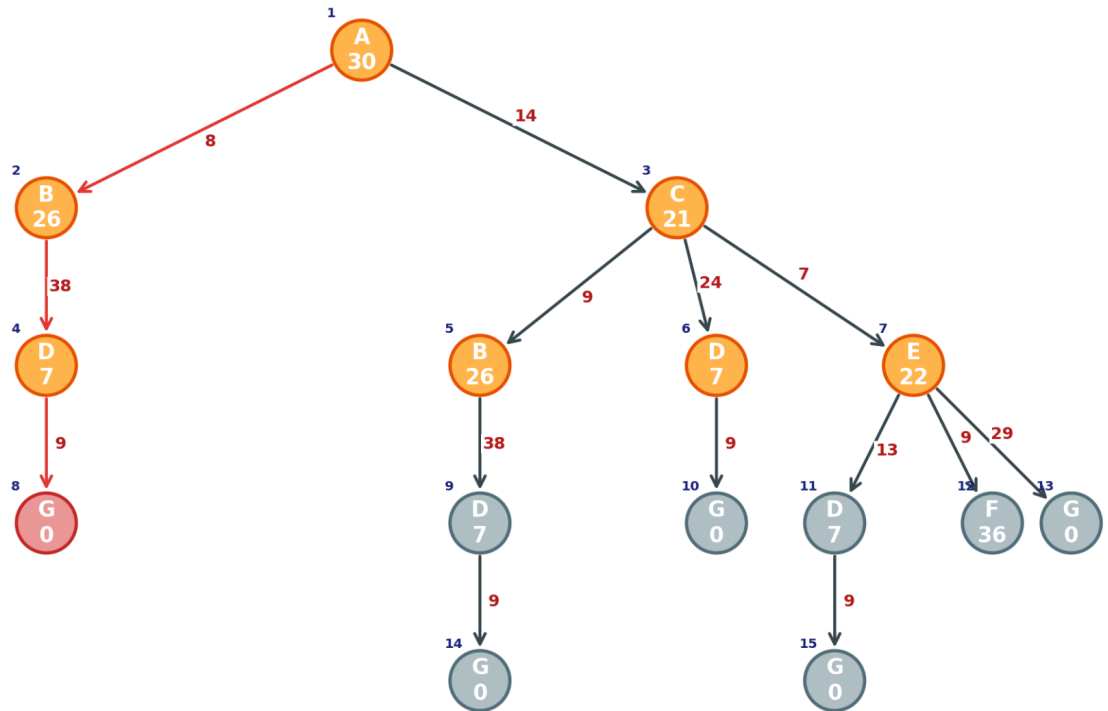


C. Na busca do nó objetivo G, que nós são expandidos usando as seguintes estratégias de busca - mostre a árvore de busca para cada caso. OBS.: empates são resolvidos expandindo-se os nós mais à esquerda.

a) Busca em largura

R: após expandir em largura, encontrei o caminho:

A -> B -> D -> G com um custo de $8+38+9 = 55$.



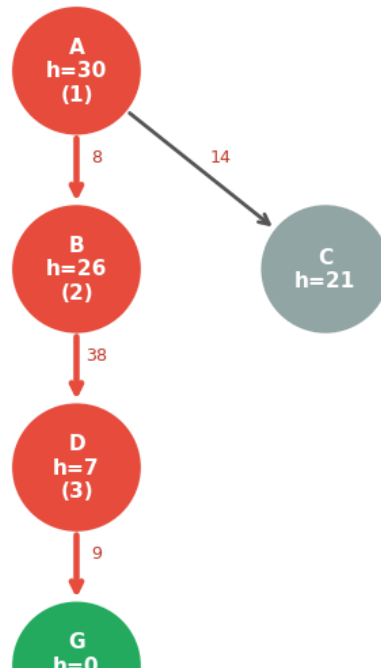
b) Busca em profundidade

R: A ordem de expansão foi A(1), B(2), D(3), G(4).

Caminho encontrado: A -> B -> D -> G, custo = $8+38+9 = 55$.

c) Busca de custo uniforme

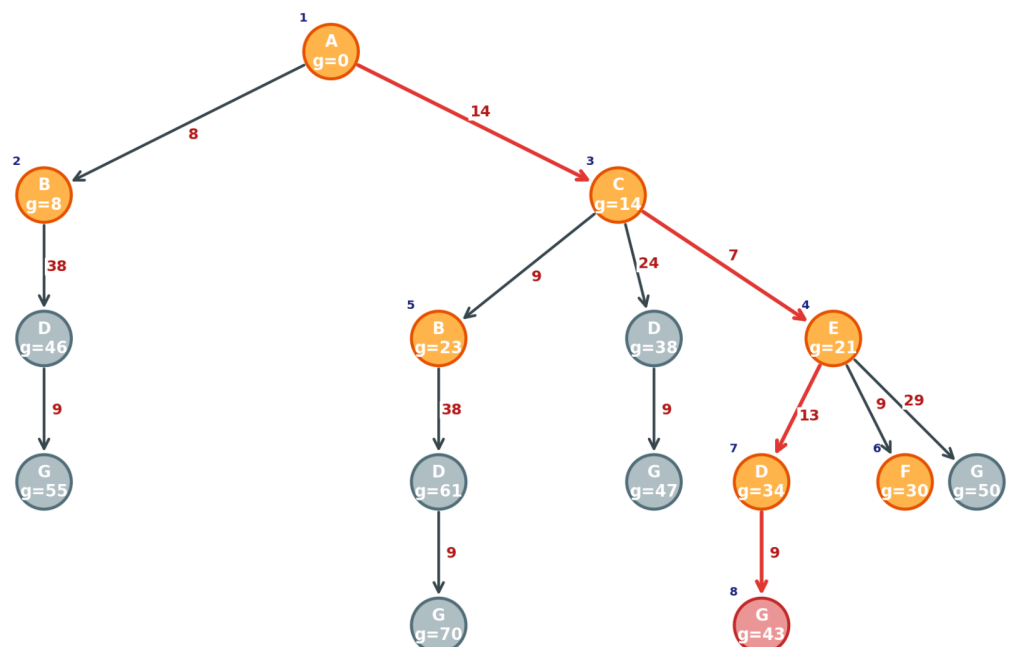
Q1C-b — DFS (Busca em Profundidade)



R: A UCS expande pelo menor custo acumulado.

Expansão: 1-A($g=0$), 2-B($g=8$), 3-C($g=14$), 4-E($g=21$), 5-C via B($g=17$, descartado), 6-D via E($g=34$), 7-F($g=50$), 8-G($g=43$).

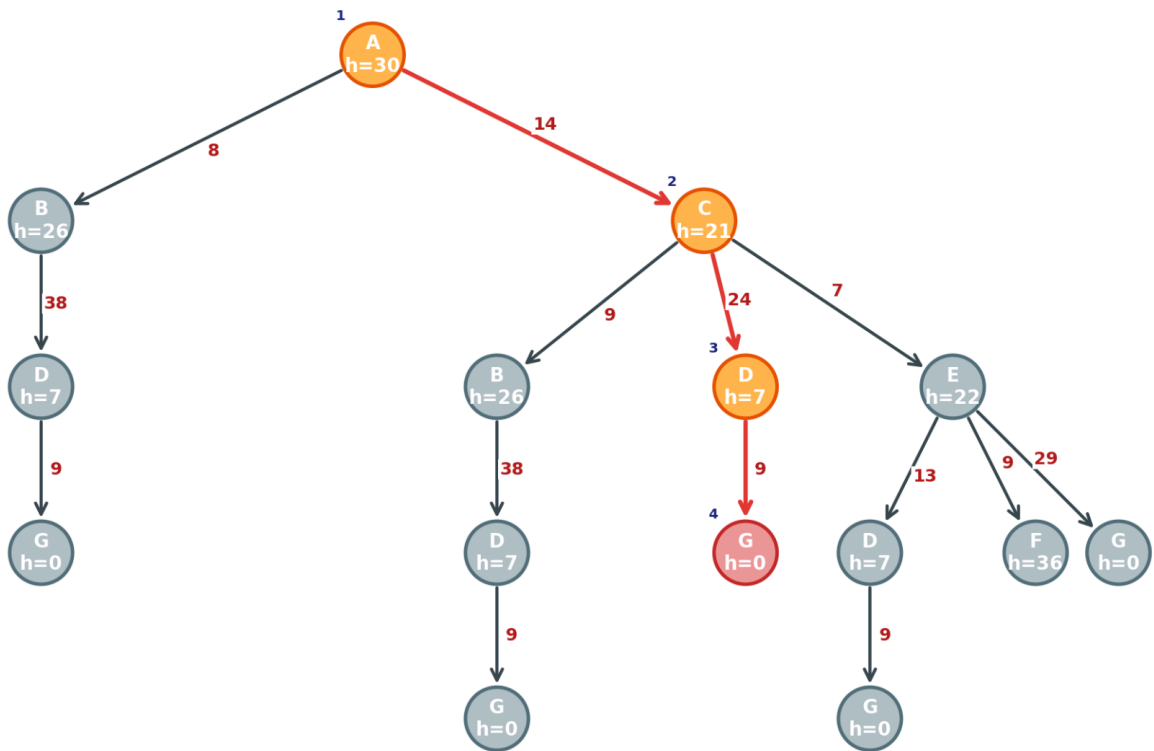
Caminho encontrado: A $\rightarrow$ C $\rightarrow$ E $\rightarrow$ D $\rightarrow$ G com um custo = $14+7+13+9 = 43$.



d) Busca Gulosa

R: A busca gulosa expande pelo menor h. Expansão foi de 1-A($h=30$), 2-C($h=21$) menor dos sucessores, 3-D($h=7$) menor dos sucessores de C, 4-G($h=0$).

Caminho encontrado: A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ G com um custo de $14+24+9 = 47$.

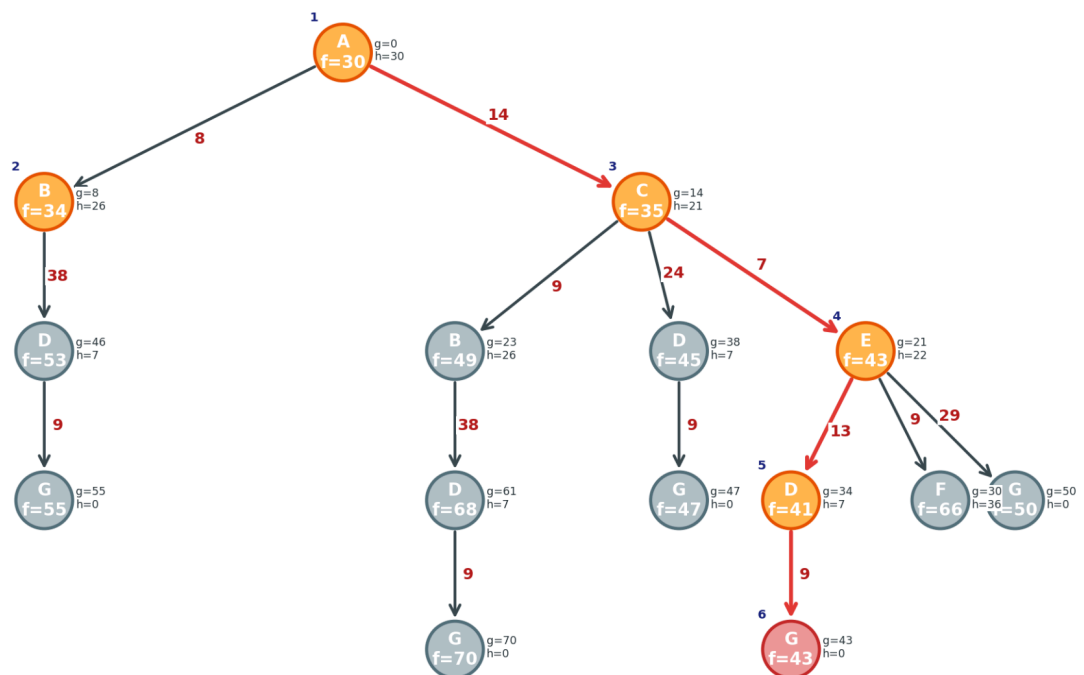


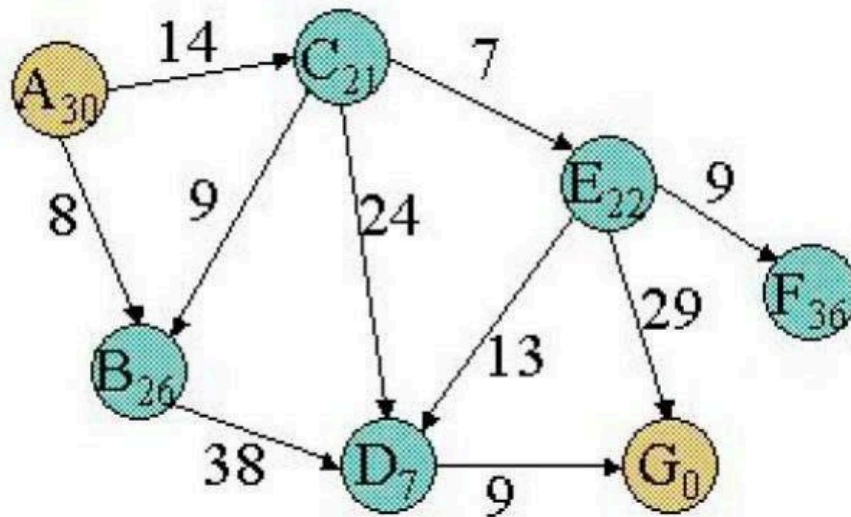
e) A*

R: O A* utiliza $f(n) = g(n) + h(n)$.

Expansão: 1-A($f=0+30=30$), 2-B($f=8+26=34$), 3-C($f=14+21=35$), 4-E($f=21+22=43$), 5-D via E($f=34+7=41$), 6-G($f=43+0=43$)

Caminho encontrado: A -> C -> E -> D -> G com um custo de $14+7+13+9 = 43$.





2. Qual é a diferença entre uma busca informada e uma busca não informada?

R: A busca não informada não usa nenhuma informação sobre o objetivo além da definição do problema, ou seja, não distingue estados mais ou menos promissores. exemplo disso são as BFS, DFS, UCS. A busca informada usa uma função heurística $h(n)$ que estima o custo do estado n ao objetivo, priorizando estados mais promissores

3. O que é uma heurística? E uma heurística admissível? E uma heurística consistente? Toda heurística consistente é também admissível?

R: Heurística é uma função que estima o custo do nó n até o objetivo, guiando a busca sem garantia de melhor caminho, a heurística admissível: $h(n) \leq h^*(n)$ para todo n , nunca superestima o custo real, ou seja, garante otimalidade do A* em busca em árvore. já a heurística consistente $h(n) \leq c(n,a,n') + h(n')$ para todo nó n e sucessor n' garante valores $f(n)$ não-decrescentes no caminho e otimalidade do A* em grafo sem reabrir nós e por fim Toda heurística consistente é admissível, mas o contrário não é necessariamente obrigatório

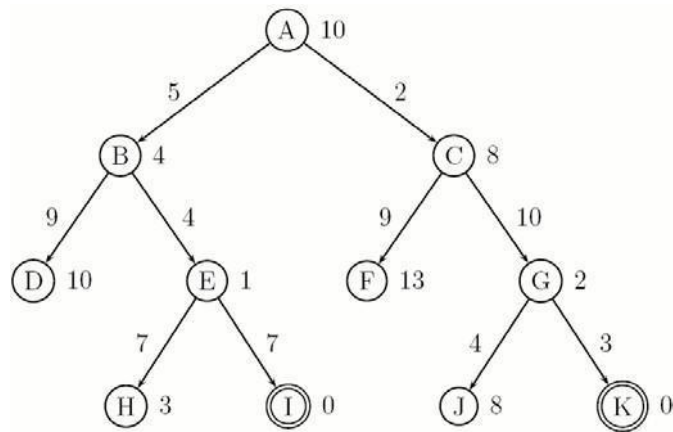
4. Quais são as condições para que a busca A* seja ótima e completa?

R: Para que o A* seja ótimo a heurística deve ser admissível em busca em árvore e em busca em grafo, deve ser consistente, pois nós expandidos não são reabertos. Para que o A* seja completo sempre encontre solução se ela existir, ou seja, o espaço de busca deve ser finito, ou os custos das ações devem ser maiores que $\epsilon > 0$, evitando ciclos infinitos de custo zero.

5. Qual é a diferença de uma busca informada para uma busca não informada?

R: A busca não informada explora o espaço de estados sem informação adicional, tratando todos os estados igualmente. Não estima qual caminho é melhor e a busca informada usa h para orientar a expansão, convergindo mais rápido ao objetivo ao evitar caminhos pouco promissores. Principais diferenças são: Seleção de nós: informada prioriza menor h não-informada expande na ordem do algoritmo largura, profundidade etc. sem critério de qualidade e a Eficiência na busca informada expande menos nós em média, ou seja, não-informada pode explorar todo o espaço antes de encontrar a solução.

6. Considere o espaço de busca a seguir. Cada nó é rotulado por uma letra. Cada nó objetivo é representado por um círculo duplo. Existe uma heurística estimada para cada dado nó (indicada por um valor ao lado do nó). Arcos representam os operadores e seus custos associados.

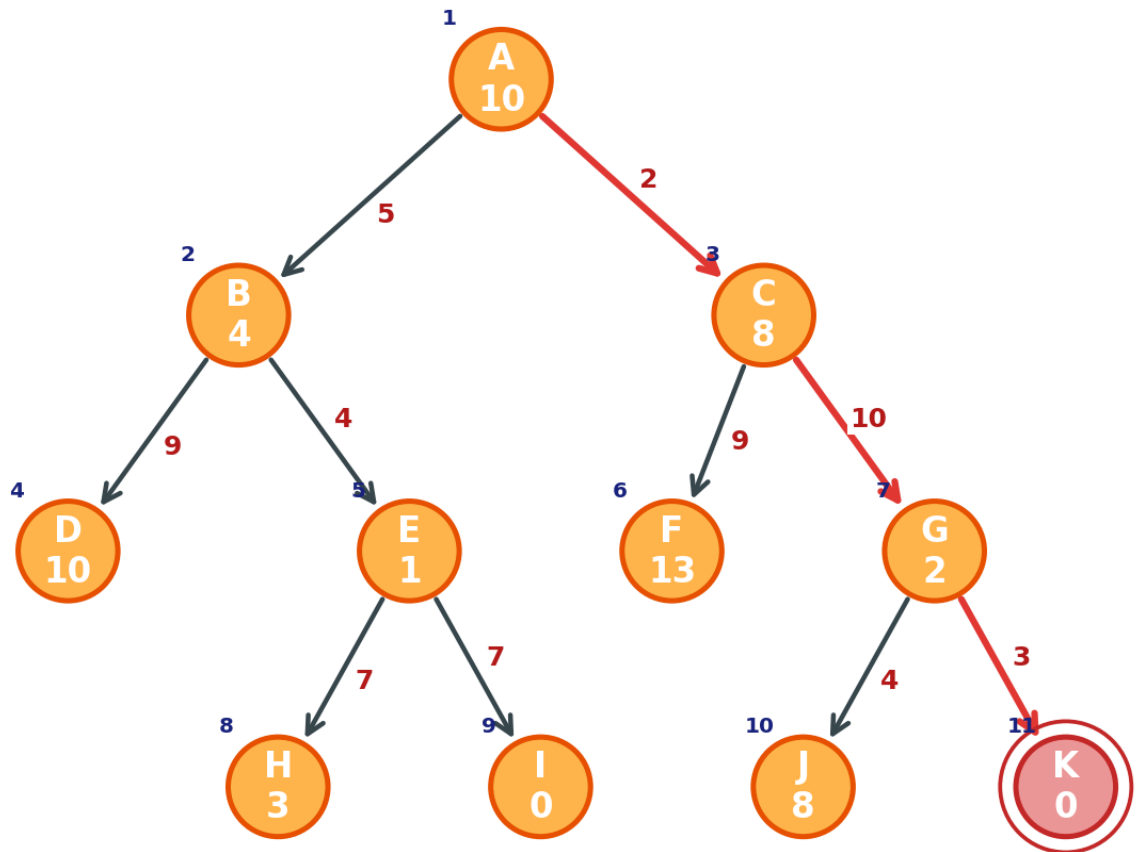


Para cada um dos algoritmos a seguir, liste os nós visitados na ordem em que eles são examinados, começando pelo nó **A**. No caso de escolhas equivalentes entre diferentes nodos, prefira o nodo mais próximo da raiz, seguido pelo nodo mais à esquerda na árvore.

a) Algoritmo de Busca em Largura;

R: A busca em largura expande nível por nível. Começando de A, primeiro expande B e C nível 1, depois D, E, F, G nível 2, e por último H, I, J, K nível 3. K é encontrado no nível 3. A ordem de expansão foi: A1, B2, C3, D4, E5, F6, G7, H8, I9, J10, K11.

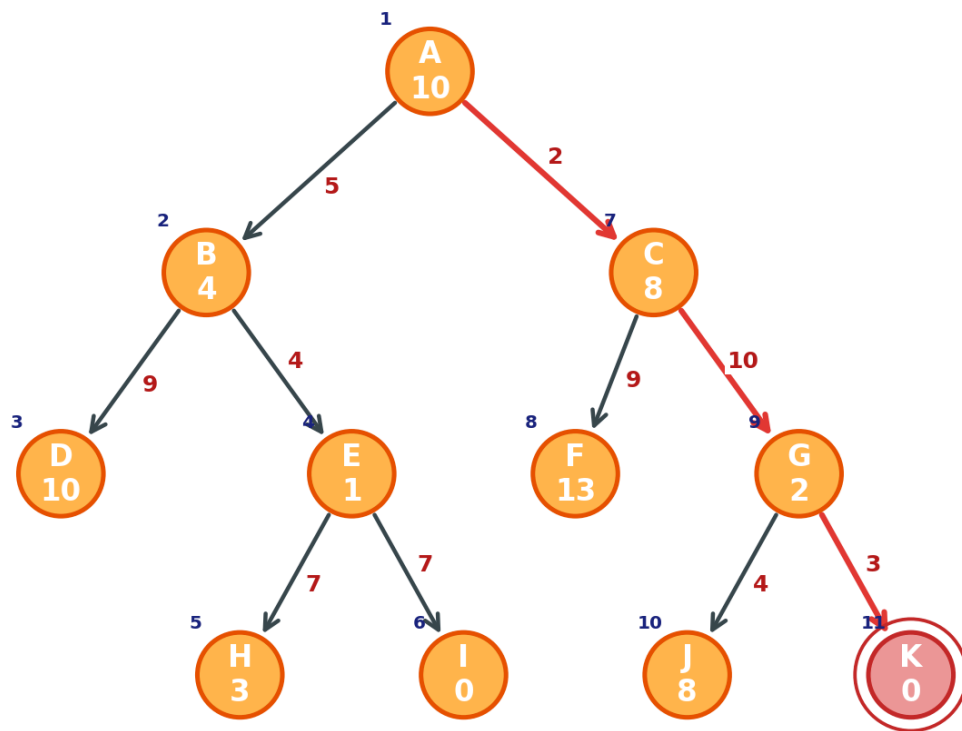
Caminho solução: $A \rightarrow C \rightarrow G \rightarrow K$, custo = $2+10+3 = 15$



b) Algoritmo de Busca em Profundidade;

R: A busca em profundidade vai sempre pelo ramo mais à esquerda até não ter mais filhos. D, H, I e F são folhas sem saída. I tem $h=0$ mas não é o objetivo, então retrocede. Ordem: A1, B2, D3, E4, H5, I6, C7, F8, G9, J10, K11.

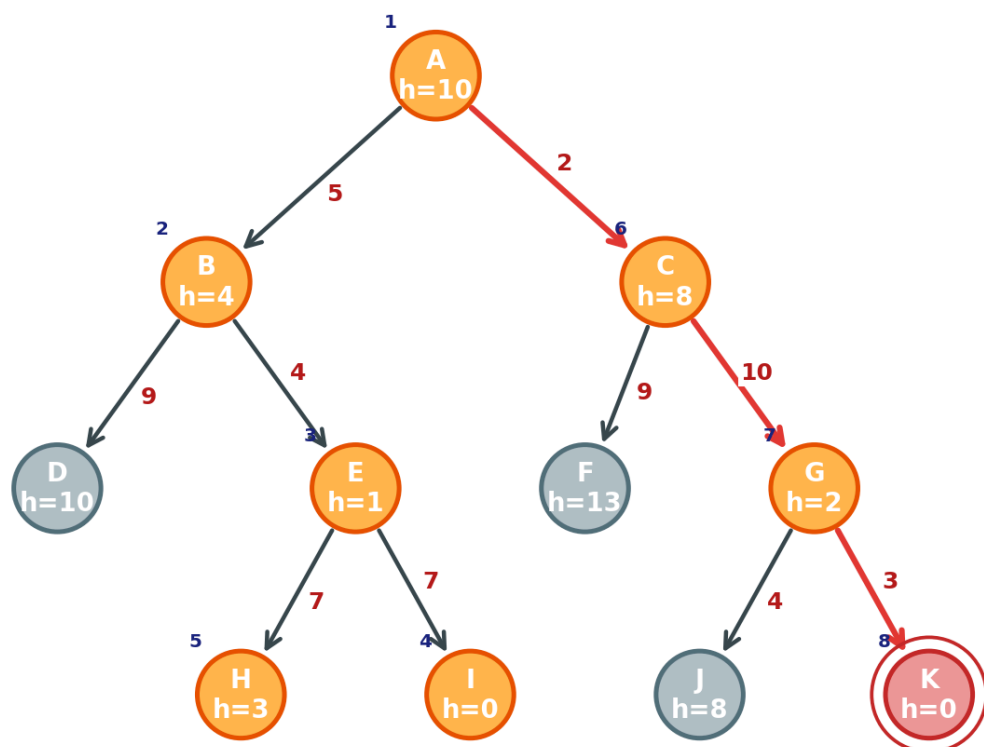
Caminho solução: $A \rightarrow C \rightarrow G \rightarrow K$, custo = 15



c) Algoritmo de Busca Gulosa;

R: Sempre expande o nó com menor $h(n)$. De A expande B no qual o $h=4$, menor. De B expande e $h=1$ e expande I $h=0$, mas I é folha e não é objetivo. Volta para H $h=3$, também folha. Agora o menor da fronteira é C $h=8$, que leva a G $h=2$, que leva a K=0. Ordem: A1, B2, E3, I4, H5, C6, G7, K8.

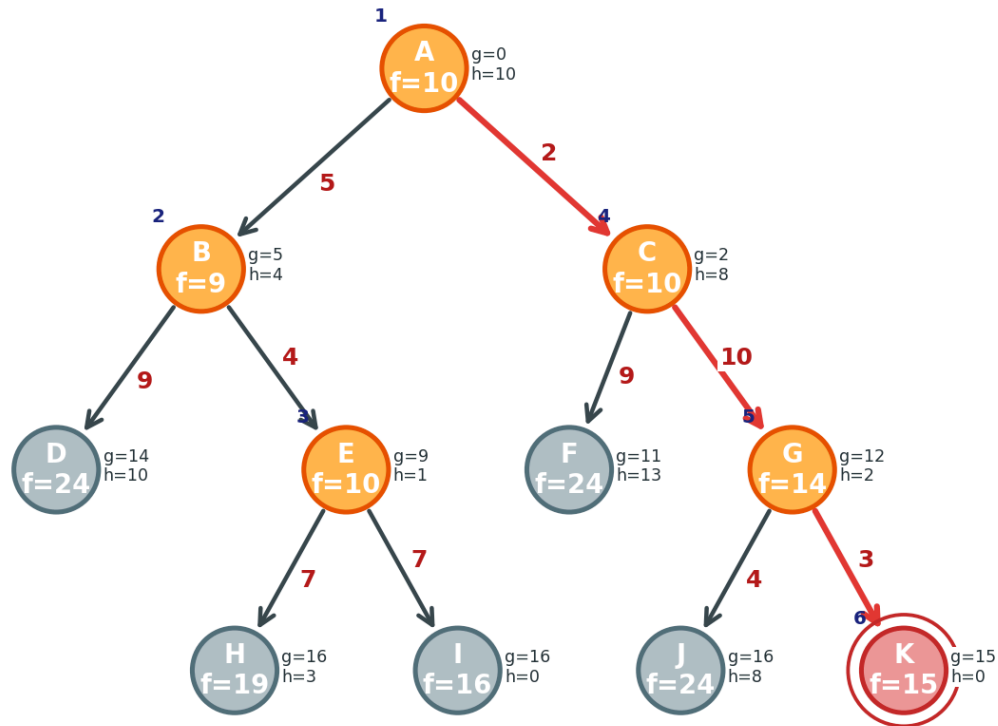
Caminho solução: $A \rightarrow C \rightarrow G \rightarrow K$, custo = 15



d) Algoritmo A*.

R: $f(n) = g(n) + h(n)$. Após expandir A, B tem $f=9$ e C tem $f=10$. Expande B. Os filhos gerados são D e E. C e E empatam, mas E tem $h=1$ menor que h de C=8, então E é expandido primeiro. Depois C, que gera G $f=14$. G gera K com $f=15$, que é expandido e é o objetivo. Ordem: A1, B2, E3, C4, G5, K6.

Caminho solução: $A \rightarrow C \rightarrow G \rightarrow K$, custo = $g(K) = 2+10+3 = 15$



7. O que significa dizer que uma heurística h_1 domina uma heurística h_2 ? O que isto quer dizer em termos de eficiência de uma busca A* usando h_1 e h_2 ?

R: Dizer que h_1 domina h_2 significa que $h_1(n)$ é maior ou igual a h_2 para n para todo nó n , sendo as duas heurísticas admissíveis. Ou seja, h_1 é mais informada que h_2 . Em termos de eficiência do A*: quando h_1 domina h_2 , o A* com h_1 expande um número igual ou menor de nós do que com h_2 . Isso acontece porque h_1 dá estimativas mais próximas do custo real, o que faz com que mais ramos sejam descartados antes. Por isso, h_1 é sempre preferível a h_2 quando ambas são admissíveis e h_1 domina h_2 , sem perder a garantia de otimalidade.

8. No escopo dos métodos de busca, explique cada item abaixo resumidamente: a) Complexidade; b) Completude; c) Quanto a ser ótimo; d) Admissibilidade;

R:

a) Complexidade do custo computacional em tempo entre nós expandidos e espaço memória para visitados, um exemplo é o BFS é o melhor em tempo e espaço.

b) Completude é a garantia de encontrar uma solução se ela existir. O BFS é completo, já o DFS pode não ser, pois consegue entrar em loop infinito quando não há verificação de ciclos.

c) Quanto a ser ótimo, é a garantia de encontrar a solução de menor custo. UCS e A* com h admissível são ótimos, enquanto o BFS só é ótimo quando os custos são uniformes.

d) Admissibilidade é quando $h(n)$ é menor ou igual a $h^*(n)$ para todo n , ou seja, a heurística nunca superestima o custo real. Isso garante que o A* encontre a solução ótima na busca em árvore.

9. Explique o método de Busca Gulosa.

R: A Busca Gulosa sempre expande o nó com menor heurística, ou seja, o aparentemente mais próximo do objetivo. Usa a função $f(n)=h(n)$ e não é garantidamente completa, pode entrar em ciclos, nem ótima, pois ignora o custo acumulado e pode seguir caminhos enganosos. É rápida quando a heurística é boa, mas sem garantias formais de qualidade.

10. Suponha um algoritmo de busca pelo melhor primeiro (best-first ou busca gulosa) em que a função objetivo é $f(n) = (2 - w) \cdot g(n) + w \cdot h(n)$. Para que valores de w este algoritmo é garantidamente ótimo? Que tipo de busca ele realiza quando $w = 0$? Quando $w = 1$? E quando $w = 2$?

R: A função $f(n)=(2-w) \cdot g(n)+w \cdot h(n)$ generaliza algoritmos de busca, ou seja, $w=0$: $f(n)=2 \cdot g(n)$ é equivalente a UCS, já o $w=1$: $f(n)=g(n)+h(n)$ é equivalente ao A*. Garantidamente ótimo se h admissível e por fim o $w=2$: $f(n)=2 \cdot h(n)$ é equivalente à Busca Gulosa Não garantindo otimalidade. O algoritmo é garantidamente ótimo para $w \leq 1$, pois nesse intervalo g de n ainda é considerado adequadamente na função de avaliação.

11. Em que sentido a busca gulosa pela melhor escolha é parecida com a busca em profundidade?

R: A busca gulosa se assemelha à DFS porque ambas tendem a seguir um único caminho profundo em direção ao objetivo antes de considerar alternativas. Na DFS, o critério é sempre o filho mais à esquerda ou mais profundo. Na Gulosa, o critério é o nó com menor $h(n)$. Em ambos os casos, pode-se acabar em um caminho longo sem saída e precisar retroceder. A diferença é que a Gulosa usa informação do domínio (h) para escolher, enquanto a DFS escolhe cegamente pela estrutura da árvore.

12. É correto afirmar que Busca pelo custo uniforme é um caso especial de A*?

R: Sim, é correto. A UCS é um caso especial do A* onde $h(n)=0$ para todos os

nós. Com $h(n)=0$, a função do A^* se reduz a $f(n)=g(n)+0=g(n)$, que é exatamente o critério da UCS: expandir sempre o nó de menor custo acumulado. A heurística nula é trivialmente admissível, por isso a UCS herda todas as garantias de otimalidade e completude do A^*

13. Qual a principal desvantagem do A^* ?

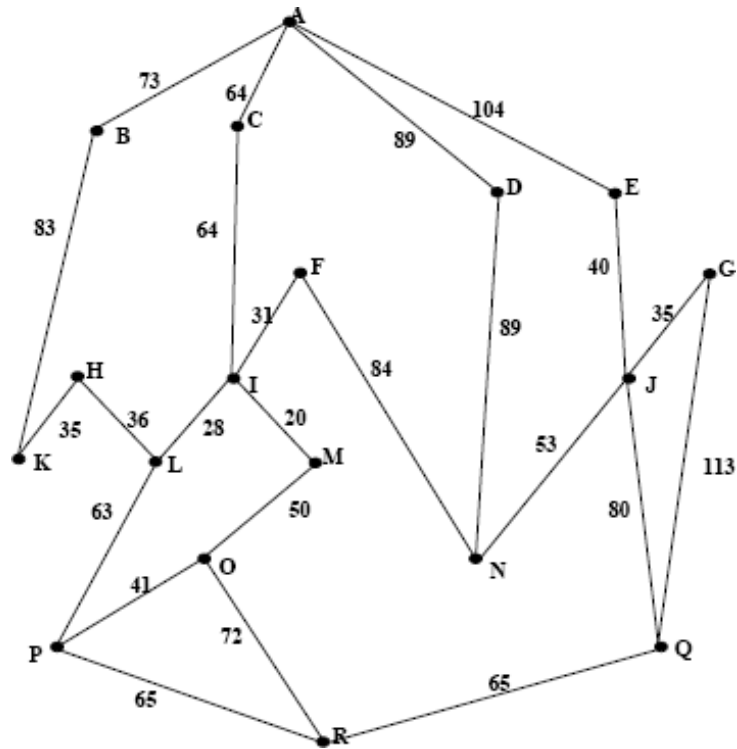
R: A principal desvantagem do A^* é o alto consumo de memória. O algoritmo mantém todos os nós gerados na fronteira e no conjunto de visitados, com complexidade de espaço exponencial em profundidade e ramificação. Para problemas grandes, isso pode tornar o A^* impraticável. Além disso, a qualidade da solução depende fortemente da heurística, um exemplo é quando h ruim o A^* lento.

14. Explique quando um método de busca é monotônico.

R: Um método de busca é monotônico quando usa uma heurística consistente, ou seja, $h(n) \leq c(n,a,n') + h(n')$ para todo nó n e sucessor n' . Isso garante que os valores $f(n)=g(n)+h(n)$ sejam não-decrescentes ao longo de qualquer caminho, evitando a necessidade de reabrir nós já expandidos. A consistência é uma condição mais forte que a admissibilidade e garante otimalidade do A^* em busca em grafo de forma eficiente.

15. Considere o seguinte mapa (fora de escala)

R: O A^* expande os nós em ordem crescente de $f(n) = g(n)+h(n)$. Começando de A, mantém uma fronteira com os nós gerados e sempre expande o de menor f . Os valores g acumulam o custo real do caminho percorrido e h é a heurística fornecida. O algoritmo para quando R é expandido, e o caminho traçado de volta até A é a solução ótima.



Usando o algoritmo A* determine uma rota de A até R, usando as seguintes funções de custo $g(n)$ = a distância entre cada cidade (mostrada no mapa) e $h(n)$ = a distância em linha reta entre duas cidades. Estas distâncias são dadas na tabela abaixo.

Em sua resposta forneça o seguinte:

a. A árvore de busca que é produzida, mostrando a função de custo em cada nó.

R: Aplicando $f(n) = g(n) + h(n)$:

A: $g=0 + h=240$ $f=240$

B: $g=73 + h=186$ $f=259$

C: $g=64 + h=182$ $f=246$

D: $g=89 + h=163$ $f=252$

E: $g=104 + h=170$ $f=274$

I: $g=128 + h=120$ $f=248$

F: $g=159 + h=150$ $f=309$

M: $g=148 + h=100$ $f=248$

L: $g=156 + h=104$ $f=260$

O: $g=198 + h=72$ $f=270$

N (via D): $g=178 + h=77$ $f=255$

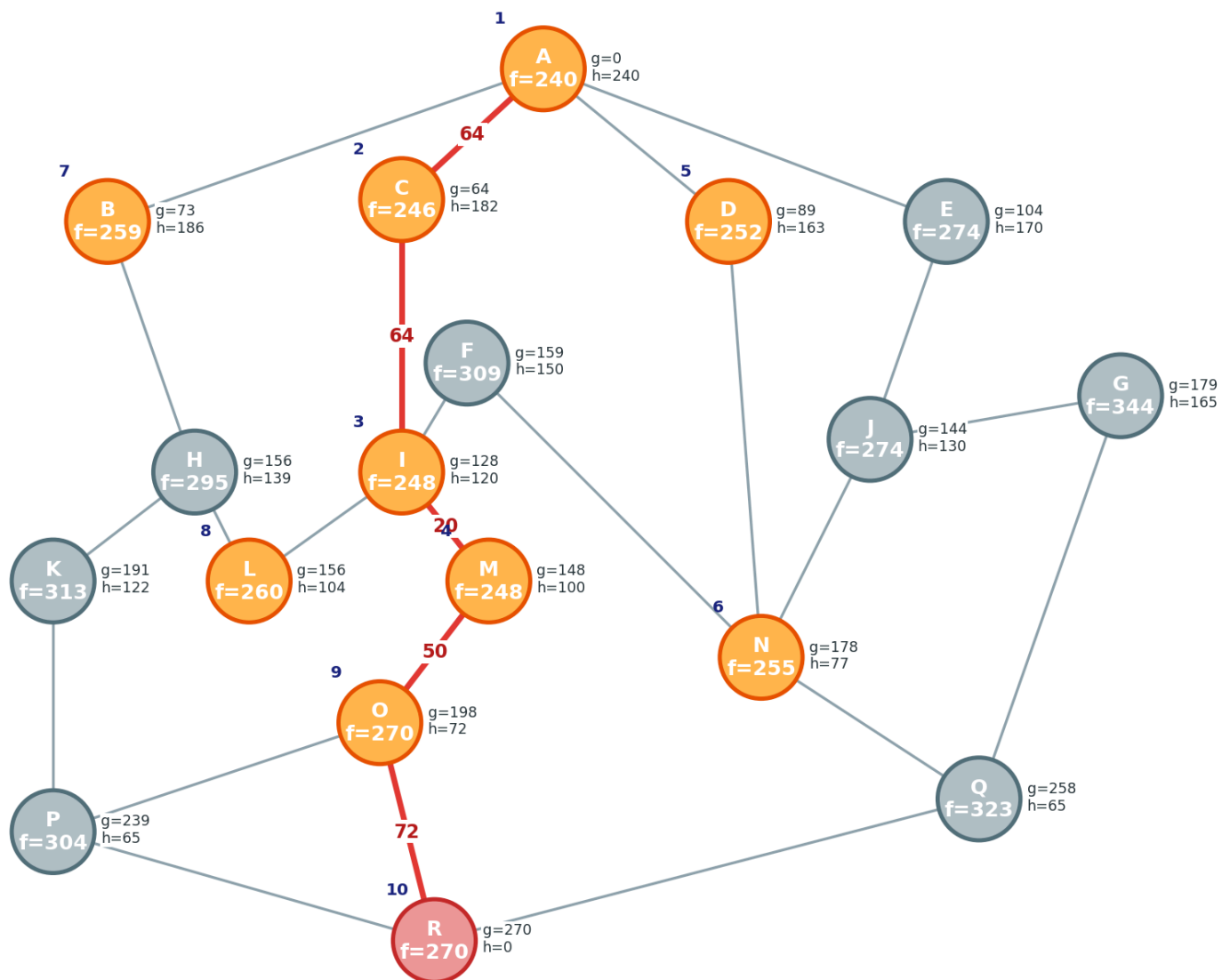
J (via N): $g=231 + h=130$ $f=361$

J (via E): $g=144 + h=130$ $f=274$

N (via J): $g=197 + h=77$ $f=274$

Q: $g=258 + h=65$ $f=323$

R: $g=270 + h=0$ $f=270$



b. Defina a ordem em que os nós serão expandidos.

R: A -> C -> I -> M -> D -> N -> B -> L -> O -> R

c. Defina a rota que será tomada e o custo total.

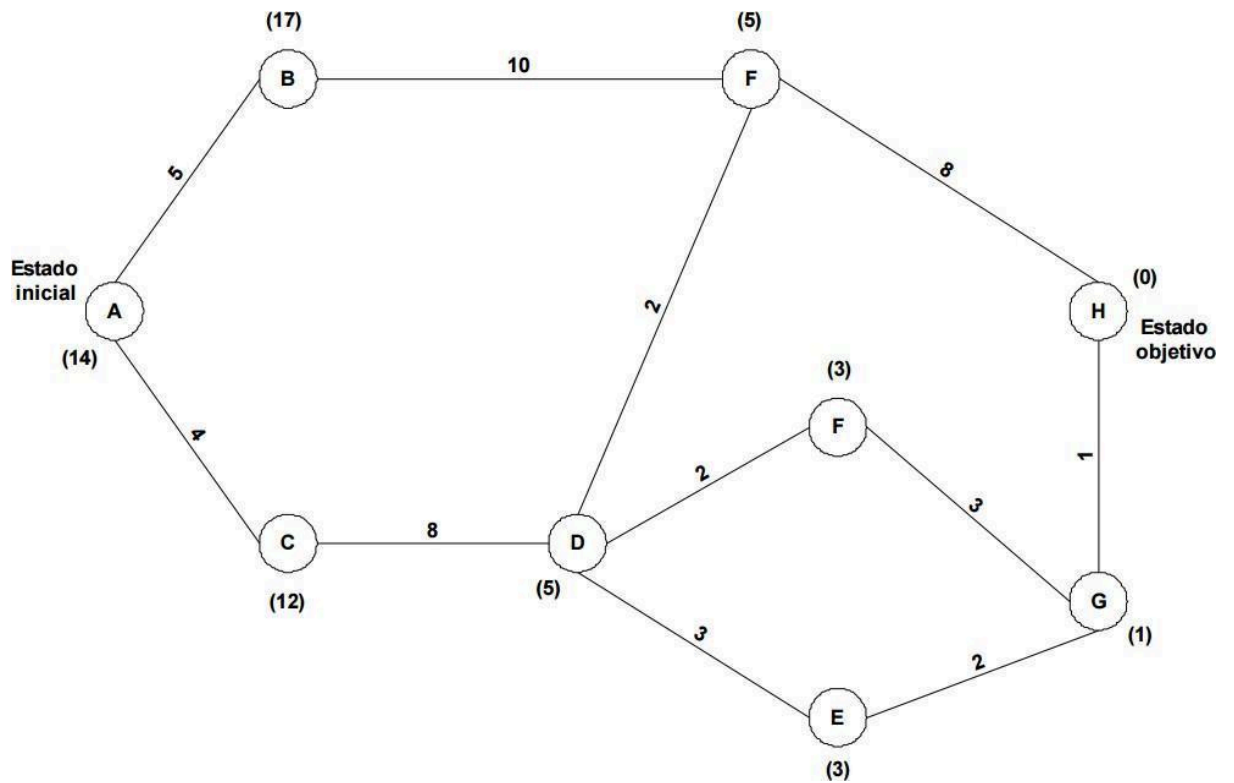
R: Rota: A -> C -> I -> M -> O -> R

Custo total: $64 + 64 + 20 + 50 + 72 = 270$

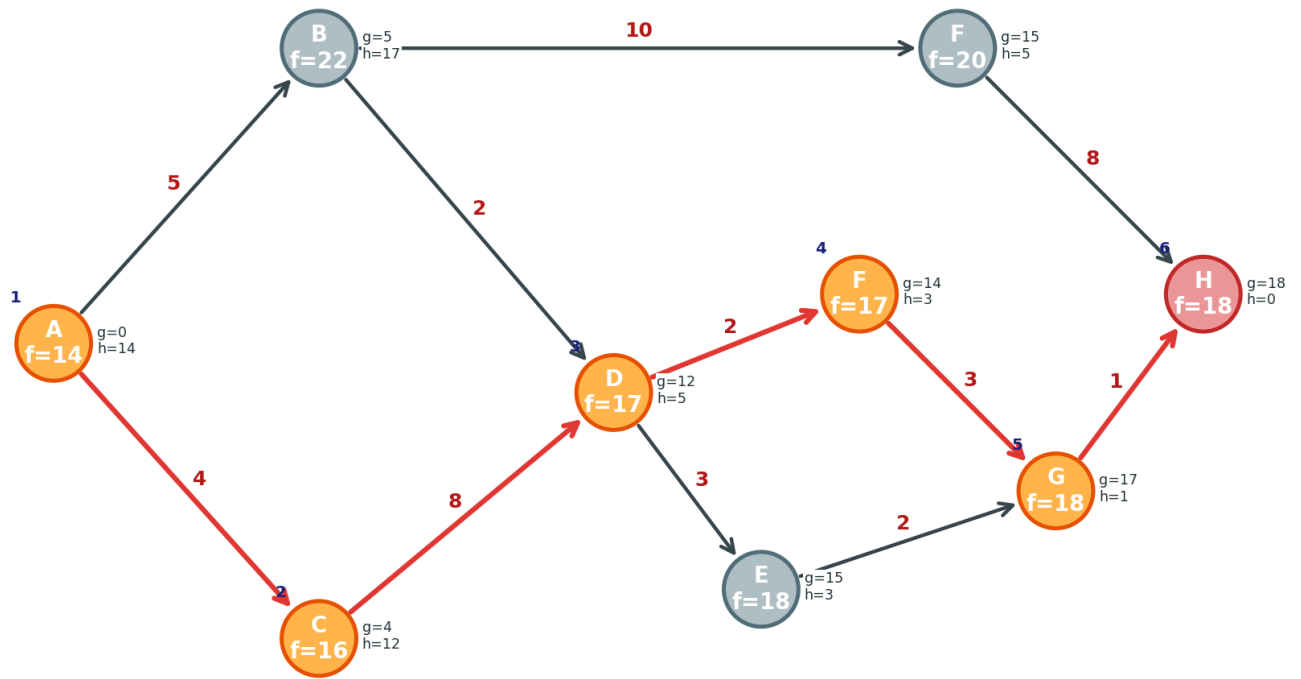
Distância em linha reta até R

A	240
B	186
C	182
D	163
E	170
F	150
G	165
H	139
I	120
J	130
K	122
L	104
M	100
N	77
O	72
P	65
Q	65
R	0

16. No grafo abaixo cada arco indica o custo do operador e entre parênteses é indicado uma estimativa do custo até o nó objetivo. Apresente as mudanças da fronteira de busca para este problema quando realizadas as buscas A* e pelo melhor primeiro (gulosa).



Busca A*



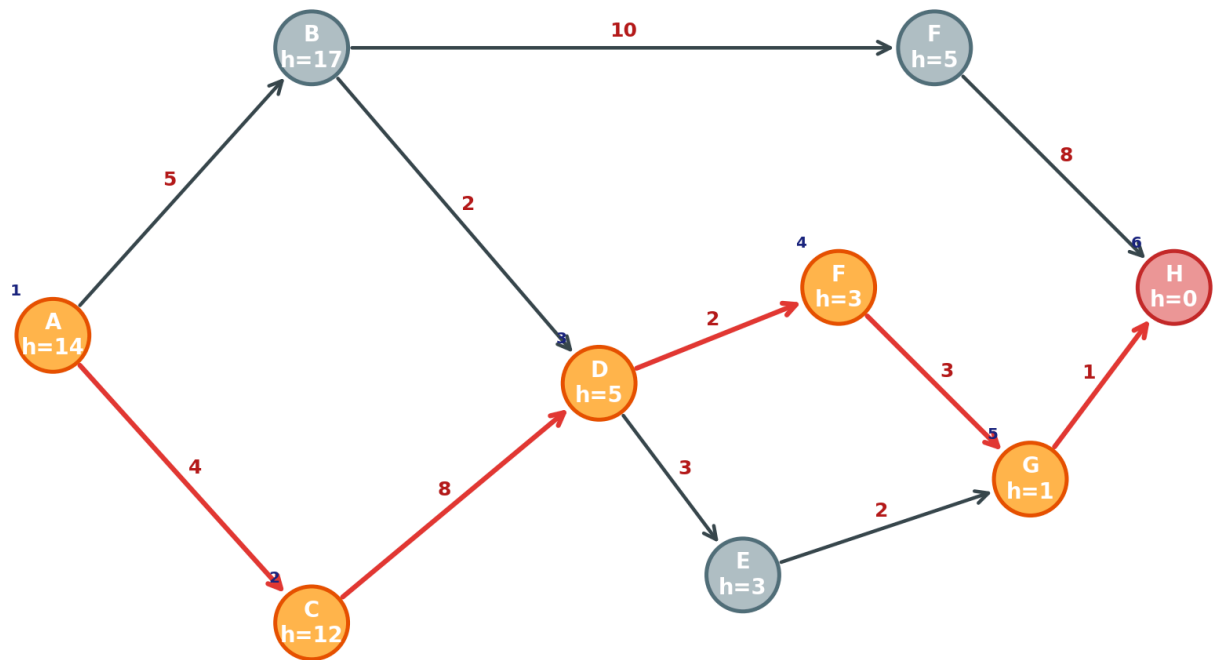
$f(n) = g(n) + h(n)$. A fronteira de busca evolui assim:

Expande A com f igual a 14, gerando B com f igual a 22 e C com f igual a 16. Em seguida expande C com f igual a 16, gerando D com f igual a 17. Depois expande D com f igual a 17, gerando F centro com f igual a 17 e E com f igual a 18. Nesse ponto, B chegaria por D com g igual a 14, pior que o g igual a 5 que B já tinha na fronteira. Expande então F centro com f igual a 17, gerando G com f igual a 18. Expande G com f igual a 18, gerando H com f igual a 18. H é o objetivo.

B foi gerado com f igual a 22, mas nunca foi expandido antes de encontrar H.

A ordem de expansão foi A, depois C, depois D, depois F centro, depois G e por fim H.

O caminho encontrado foi A para C, C para D, D para F, F para G e G para H, com custo total de 18, somando 4 mais 8 mais 2 mais 3 mais 1.



Caminho: A - C - D - F - G - H, custo = 4+8+2+3+1 = 18

Os dois algoritmos chegaram ao mesmo caminho. Vale notar que o caminho $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G \rightarrow H$ custaria $5+2+2+3+1=13$, mas como $h(B)=17$ superestima o custo real (que é 8), a heurística não é admissível neste grafo e o A^* não garante a solução ótima.

17. Imagine um robô aspirador que se encontra em uma sala quadrada dividida em quatro posições. A todo momento o robô ocupa um desses quadrados e tem uma das seguintes orientações: norte, sul, leste, oeste. Em certos quadrados, pode ter sujeira. Existem três ações que o robô pode realizar, todas com um custo de 10:

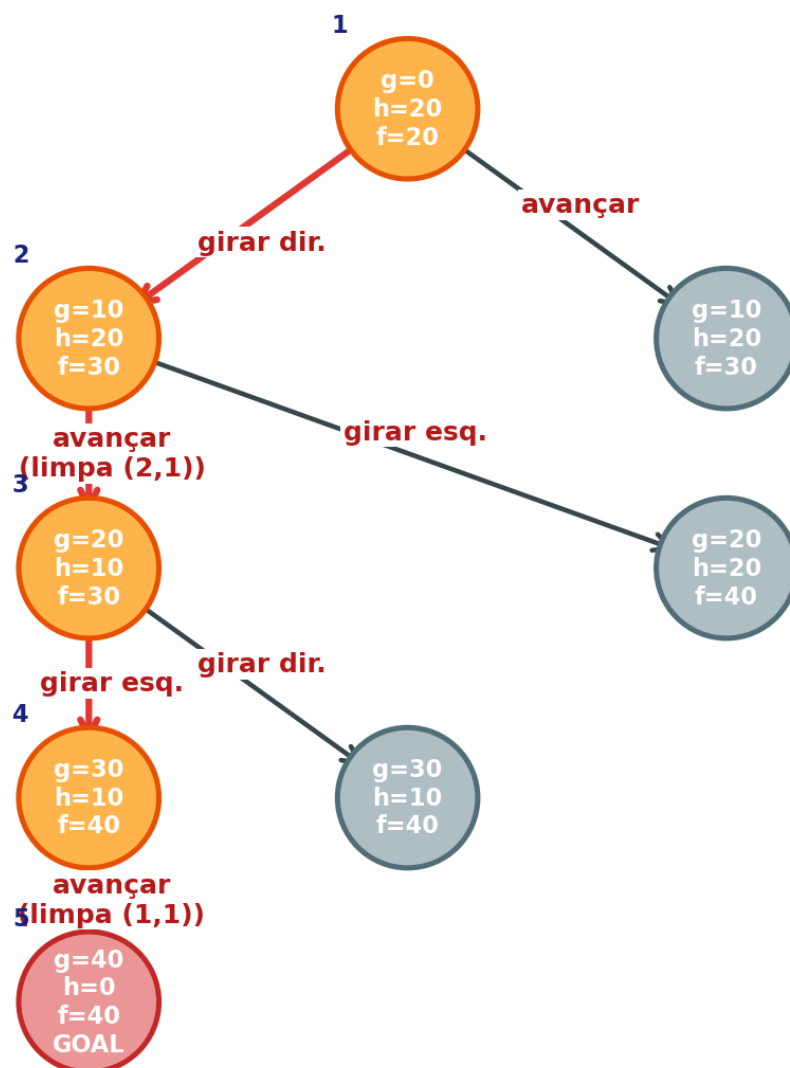
- Avançar para a posição que está na sua frente. No estado resultante, o robô se encontra na posição que está na sua frente e aspira a sujeira, se tiver sujeira nessa nova posição.
- Efetuar uma rotação de 90 graus para a direita.
- Efetuar uma rotação de 90 graus para a esquerda.

O estado final é atingido quando não existe nenhuma posição com sujeira.
 Suponhamos um estado inicial com sujeira nas posições (1,1) e (2,1), e o robô na posição (2,2) orientado na direção oeste:

	1	2
1	*	*
2		◁

ESTADO INICIAL

a) Seja a seguinte heurística admissível: $h_1(n) = 10 \times N_s$, onde N_s é o número de quadrados que contêm sujeira. Mostre como está a árvore de busca no momento em que o algoritmo A* encontra a solução e enumere os nodos na ordem em que eles foram visitados. Para cada nodo, indique também qual é o seu valor $f(n)$.



$h_1(n) = 10 \times N_s$, onde N_s é o número de quadrados sujos.

Estado inicial em (2,2) virado para Oeste, com sujeira em (1,1) e (2,1). O custo g é 0, a heurística h_1 é 20 e o f é 20.

Girar para a direita, ficando em (2,2) virado para Norte. O custo g sobe para 10, h_1 continua 20 e o f fica 30.

Avançar e limpar (2,1), chegando em (2,1) virado para Norte, com sujeira só em (1,1). O g é 20, h_1 é 10 e o f é 30.

Avançar a partir do estado inicial, indo para (1,2) virado para Oeste, ainda com as duas sujeiras. O g é 10, h_1 é 20 e o f é 30.

Girar para a esquerda a partir do estado inicial, ficando em (2,2) virado para Sul, com as duas sujeiras. O g é 10, h_1 é 20 e o f é 30.

Girar para a esquerda a partir do terceiro estado, ficando em (2,1) virado para Oeste, com sujeira em (1,1). O g é 30, h_1 é 10 e o f é 40.

Por fim, avançar e limpar (1,1). O g chega a 40, h_1 vai a 0 e o f fica 40. Esse é o objetivo.

b) Mostre que $h_1(n)$ é admissível.

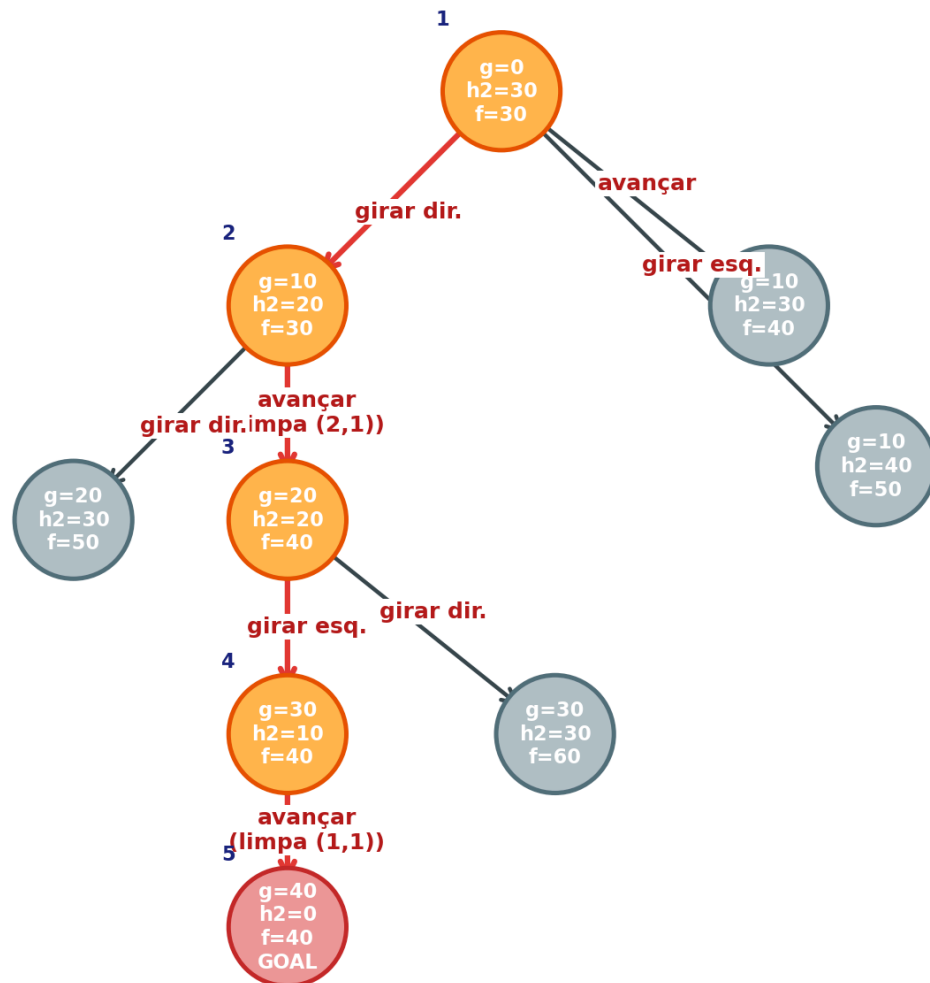
Admissibilidade de h_1 e que cada quadrado sujo exige pelo menos uma ação, avançar com um custo 10 para ser limpo. Logo, para todo n , ou seja, h_1 nunca superestima h_1 é admissível.

c) Proponha uma heurística $h_2(n)$, admissível, que domina $h_1(n)$. Mostre como está a árvore de busca no momento em que o algoritmo A^* encontra a solução e enumere os nodos na ordem em que eles foram visitados. Para cada nodo, indique também qual é o seu valor $f(n)$.

R: A heurística h_2 é definida como 10 vezes N_s mais 10 vezes $R_{\min}$, onde N_s é o número de quadrados sujos e $R_{\min}$ é o número mínimo de rotações para o robô ficar de frente para o quadrado sujo mais próximo, sendo zero se ele já estiver de frente.

h_2 domina h_1 : como $R_{\min}$ é sempre maior ou igual a zero, temos que h_2 é maior ou igual a h_1 para todo n .

h_2 é admissível: o robô sempre precisa de pelo menos $R_{\min}$ rotações antes de poder avançar até o primeiro quadrado sujo, além dos N_s avanços necessários para limpar todos eles. Por isso, o custo real $h^*(n)$ é sempre maior ou igual a 10 vezes N_s mais 10 vezes $R_{\min}$, que é exatamente h_2 , para todo n .



Todos expandidos com h2 na ordem de visita:

Em (2,2) virado para Oeste, com sujeira em (1,1) e (2,1), o g é 0, h2 é 30 e f é 30. R_min é 1, pois precisa girar para o Norte.

Em (2,2) virado para Norte, o g é 10, h2 é 20 e f é 30. R_min é 0, já que (2,1) está à frente.

Em (2,1) virado para Norte, com sujeira em (1,1), o g é 20, h2 é 20 e f é 40. R_min é 1, pois precisa girar à esquerda.

Em (2,1) virado para Oeste, o g é 30, h2 é 10 e f é 40. R_min é 0, já que (1,1) está à frente. Avançando e limpando (1,1), o g chega a 40, h2 a 0 e f a 40, alcançando o objetivo.

O caminho solução é girar à direita, avançar, girar à esquerda e avançar, com custo total de 40. Aqui R_min é o número mínimo de rotações para o robô ficar de frente para o quadrado sujo mais próximo, sendo zero se já estiver de frente.

h2 domina h1, pois R_min é sempre maior ou igual a zero, então $h2(n)$ é maior ou igual a $h1(n)$ para todo n. E h2 é admissível, pois o robô sempre precisa de pelo menos R_min rotações antes de avançar para o primeiro sujo, além dos Ns avanços. Logo $h^*(n)$ é sempre maior ou igual a $h2(n)$

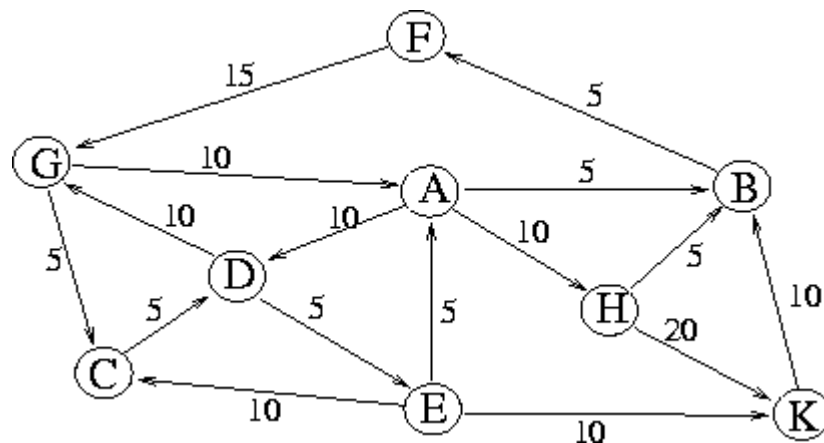
Comparação entre as duas árvores:

Com h_1 , o A^* expandiu 6 nodos. Os estados (1,2) Oeste e (2,2) Sul têm f igual a 30, então entram na fronteira junto com (2,2) Norte e precisam ser gerados e comparados antes de seguir.

Com h_2 , esses estados passam a ter f igual a 40 e 50, porque h_2 conta as rotações mínimas além das limpezas. Assim eles ficam atrás do caminho correto na fila e nunca são expandidos, e a árvore fica com apenas 4 nodos.

Comparando as duas, h_2 gerou e expandiu menos nodos e chegou ao mesmo resultado ótimo de custo 40. Isso confirma que h_2 é melhor: por dominar h_1 , guia o A^* de forma mais eficiente sem perder a otimalidade.

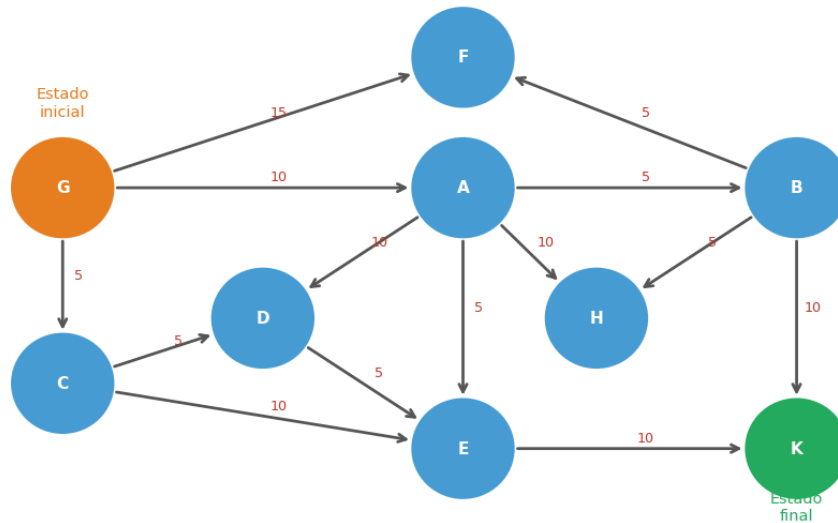
18. Suponha um problema que pode ser representado como um espaço de 9 estados (designados pelas letras A até K). Considere o seguinte grafo, que representa os sucessores possíveis para cada estado:



Os nodos representam os estados e as arestas o custo para passar de um estado a outro estado. A direção da flecha indica o estado resultante. Por exemplo, é possível atingir o estado A a partir do estado G com um custo de 10.

Suponha agora que G é o estado inicial e K o estado final. Mostre a a sequência dos nodos visitados (ordem de visita) e os nodos que estão esperando para ser visitados no momento em que se encontra uma solução, em cada uma das seguintes buscas:

Q18 — Grafo do problema (G→K)

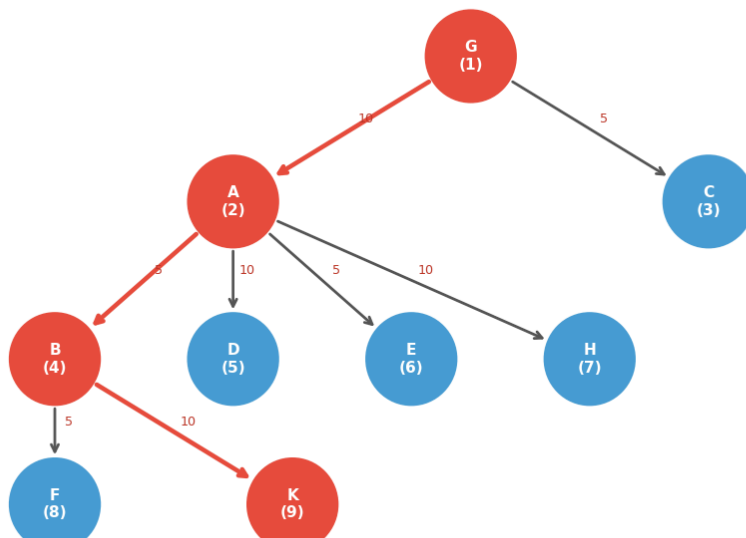


a) Busca em largura.

No nível 0 está G. No nível 1, A com g igual a 10 e C com g igual a 5. No nível 2, B com g igual a 15 via A, D com g igual a 15 via C, E com g igual a 15 via A, H com g igual a 20 via A e E com g igual a 15 via C. No nível 3, F com g igual a 20 via B e K com g igual a 25 via B. K é encontrado.

A ordem de expansão foi G, A, C, B, D, E, H, F, K. O caminho solução foi G para A, A para B e B para K, com custo total de 25, somando 10 mais 5 mais 10. Os nodos que ficaram aguardando na fronteira foram D, E, H e F.

Q18a — Busca em Largura (BFS) de G até K

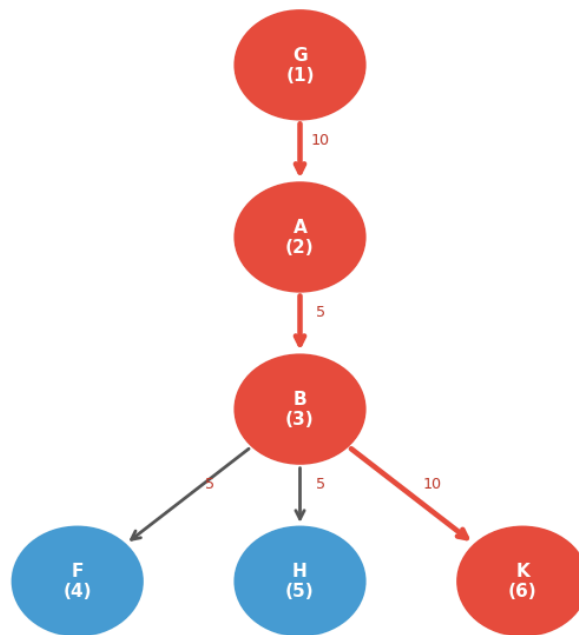


b) Busca em profundidade.

DFS vai sempre pelo filho mais à esquerda. Começa em G, vai para A, depois B, depois F, que é folha, então retrocede. Em seguida vai para H e por fim K, encontrando o objetivo.

A ordem de expansão foi G, A, B, F, H e K. O caminho solução foi G para A, A para B e B para K, com custo total de 25, somando 10 mais 5 mais 10. Os nodos que ficaram aguardando quando K foi encontrado foram C, D e E.

Q18b — Busca em Profundidade (DFS) de G até K



c) Busca com o algoritmo A*, utilizando a seguinte função heurística: $h(A) = h(C) = h(E) = h(F) = h(G) = 10$, $h(B) = 20$, $h(D) = 5$ e $h(H) = h(K) = 0$. Para cada nodo da árvore, indique o seu valor $f(n)$.

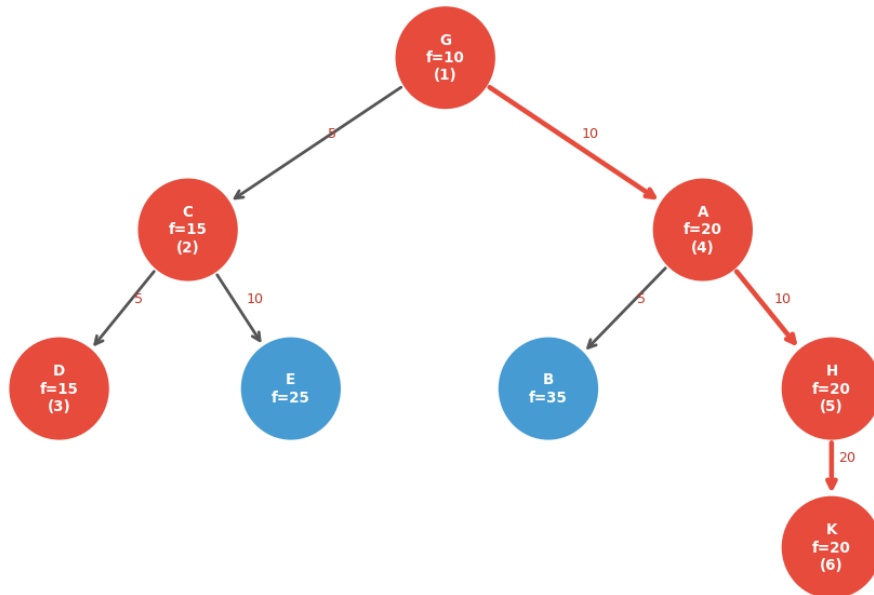
R: A* usando a heurística h no qual G, o f é 0 mais 10, igual a 10.

Expande G, gerando C com f igual a 15, sendo 5 mais 10, e A com f igual a 20, sendo 10 mais 10. Expande C com f igual a 15, gerando D com f igual a 15 e E com f igual a 25. Expande D com f igual a 15, que geraria E com f igual a 30, pior do que o já existente. Expande A com f igual a 20, gerando B com f igual a 35, E com f igual a 25 que já estava na fronteira e H com f igual a 20. Expande H com f igual a 20, gerando K com f igual a 20. K é o objetivo.

A ordem de expansão foi G, C, D, A, H e K. O caminho solução foi G para A, A para H e H para K, com custo total de 40, somando 10 mais 10 mais 20.

Vale notar que $h(B)$ é igual a 20 e não é admissível, já que o custo real $h^*(B)$ é 10. Por isso, nesse caso o A* não garante encontrar a solução ótima.

Q18c — A* de G até K ($h(B)=20$ não admissível!)



d) Dessas três buscas, qual é a melhor para esse problema específico? Justifique.

Para este problema específico, a Busca em Largura e a Busca em Profundidade são mais adequadas, pois ambas encontram o caminho ótimo G para A, A para B e B para K, com custo 25, e com menos expansões.

O A* com as heurísticas fornecidas não garante o ótimo, porque h de B é igual a 20 e o custo real h^* de B é 10, o que viola a admissibilidade. Com uma heurística admissível, o A* seria melhor, pois expandiria menos nós e ainda encontraria o ótimo. Portanto, a melhor opção neste caso específico é a BFS, que é completa e ótima para este grafo.

19. Considere o seguinte quebra-cabeça constituído de 3 peças brancas de um lado, 3 peças pretas do outro lado, e uma posição vazia:



O objetivo é inverter a ordem das peças, isto é, ter todas as peças brancas do lado esquerdo e as pretas à direita. E isso ao menor custo possível. Para obter esse estado existem três movimentos possíveis:

1. Deslizar uma peça no lugar vazio se ele é contíguo a essa peça (custo = 1).
2. Uma peça pode pular encima de outra peça para alcançar o lugar vazio (custo = 1).
3. Uma peça pode pular encima de duas outras peças para alcançar o lugar vazio (custo = 2).

Proponha uma heurística admissível para resolver esse problema com A*.

R: Heurística proposta de h é o número de peças que estão no lado errado do tabuleiro. Por exemplo, conta-se cada peça preta ainda no lado esquerdo, das brancas, e cada peça branca

ainda no lado direito, das pretas.

Admissibilidade: cada peça fora do lugar precisa de pelo menos um movimento para ser reposicionada, já que deslizar custa 1, pular sobre uma peça custa 1 e pular sobre duas custa 2. Assim, o custo real $h^*(n)$ é sempre maior ou igual ao número de peças fora do lugar, portanto $h(n)$ é menor ou igual a $h^*(n)$. No estado inicial, todas as 6 peças estão no lado errado, então h é igual a 6.

Q19 — Heurística admissível: peças no lado errado

